

Workflow Orchestrator

Product requirements document — v2 · multi-tenant agentic platform

Executive summary. A SaaS platform that lets enterprise ops teams describe routine multi-step jobs in natural language and run them reliably across their own systems. The platform accepts intent (“fetch X, compare with Y, mail the result to Z”), interactively clarifies missing details, compiles intent into a validated DAG of pre-approved capabilities, and executes that DAG durably with retries, observability, and per-tenant isolation. Workers run inside the customer’s network so sensitive data never crosses tenant boundaries.

Document scope

This document covers the v2 release. It defines the business case, target personas, agentic LLM behavior, capability registry and version control, multi-tenant architecture, the message broker decision, the Temporal-based orchestration layer, target SLAs, security posture, client-side requirements, and the algorithms (in pseudocode) that govern planning, validation, and execution.

Out of scope for v2

- Tenant-authored custom adapters (deferred to v3 — the marketplace).
- Human-in-the-loop approval gates inside running workflows (v3).
- Cross-tenant workflow composition.
- Federated multi-region active-active (v2 ships single-region active with warm DR).
- On-prem control-plane installation (v2 is SaaS-only; only the agent runs at the customer site).

1. Glossary & knowledge base

Definitions used throughout this document. Each term is precise — if a section uses one of these words, it means exactly this.

Term	Definition
Capability	A registered unit of work the platform can perform (e.g. http.fetch, gcs.upload, file.diff). Versioned, signed, and bound to a sandboxed adapter image.
Capability registry	The authoritative catalogue of capabilities. The LLM reads it, the validator checks against it, and the orchestrator dispatches by it. Source of truth.
DAG / plan	A directed acyclic graph of capability invocations with bound parameters and wired inputs/outputs. The compiled, validated form of user intent.
Adapter	The runtime artifact (signed OCI image) that implements a single capability version. Loaded by the worker agent into a sandbox per execution.
Worker agent	A thin runtime deployed on or near the systems that should perform work. Pulls tasks over outbound TLS, executes adapters in a sandbox, emits status events.
Slot	A required parameter for a capability that the LLM must populate from user intent or by clarification (e.g. source URL, destination path, recipient list).
Clarification turn	An exchange where the agentic planner asks the user for missing or ambiguous slots before producing a DAG. Bounded by MAX_CLARIFY_TURNS (default 3).
Repair turn	An exchange where validation errors from a draft DAG are fed back to the LLM for correction. Bounded by MAX_REPAIR_TURNS (default 2).

Term	Definition
Idempotency key	A deterministic hash of a step's logical inputs used to suppress duplicate side effects on retry. Computed per (workflow, step, attempt-stable inputs).
Tenant	A logical isolation boundary. Owns its own queues, secrets, audit trail, capability allowlist, agent fleet, and policy configuration.
Saga / compensation	A reverse-action invoked when a non-idempotent step fails irrecoverably (e.g. delete a partially uploaded blob).
Vended credential	Short-lived, scoped credential issued just-in-time to an adapter for a single step (e.g. STS token valid for 15 min, scoped to one bucket prefix).

2. Business PRD

2.1 Problem statement

Enterprise operations teams run thousands of recurring multi-step procedures across heterogeneous systems — downloading vendor files, syncing them to cloud storage, comparing against baselines, generating reports, and routing alerts. These procedures live today in cron jobs, ad-hoc shell scripts, and tribal knowledge. They break silently, retry incorrectly, scatter credentials across hosts, and resist audit. When the engineer who wrote the script leaves, the procedure becomes a liability.

We want to replace this with a system where ops engineers describe the procedure in natural language and the platform handles execution, retry, isolation, and audit. Critically, the LLM must behave agentially: when the intent is incomplete (“upload it to GCS” — which bucket?), it should clarify rather than fail or guess.

2.2 Target personas

Persona	Goals	Pain points today
Ops engineer	Automate routine procedures without writing or maintaining scripts.	Brittle cron, stale credentials, no observability, on-call pages at 3am.
Tenant admin	Govern what their team can run; manage allowlists, recipients, quotas.	No central control over what scripts run where, with what permissions.
Security / compliance	Audit every action, restrict egress, enforce least-privilege.	Scripts run with broad service-account credentials, audit gaps.
Platform engineer	Add new integrations (capabilities) without each tenant rebuilding pipelines.	Reusable building blocks don't exist; everyone writes their own glue.

2.3 Value propositions

- **Time to first automation in minutes, not days.** Describe in NL, confirm the plan, run.
- **Reliable execution.** Durable retries, idempotency, exactly-once side effects, structured failure modes.
- **Defensible security boundary.** Workers run in customer network; data never leaves; per-tenant CMEK; signed adapters; OPA-enforced policies; complete audit.
- **Operational leverage for the platform team.** Add a capability once; every tenant can use it instantly with NL.
- **Compliance-ready.** SOC 2, ISO 27001, GDPR-aligned by design. Every workflow submission, plan, decision, and execution is signed and replayable.

2.4 Success metrics (12-month targets)

Metric	Target	Measurement
Time-to-first-workflow	< 10 minutes from signup	Onboarding telemetry
Workflow plan acceptance rate	≥ 85% of plans accepted on first or second clarification	Plan-state tracking
Step success after retries	≥ 99.5%	Orchestrator metrics
Tenant retention (annual)	≥ 95%	Billing / contract data
NPS from ops engineers	≥ 40	Quarterly survey
Cross-tenant data incidents	0	Audit + SIEM
Mean tenants per platform engineer	≥ 50	Internal load distribution

2.5 Pricing model (assumptions, not committed)

Per-workflow execution + per-active-agent seat, with a free tier capped at 100 executions/month. Enterprise tier adds dedicated orchestrator shards, custom SLAs, and SSO/SCIM. Pricing detail is owned by the commercial team and is not a v2 engineering deliverable.

3. Functional requirements

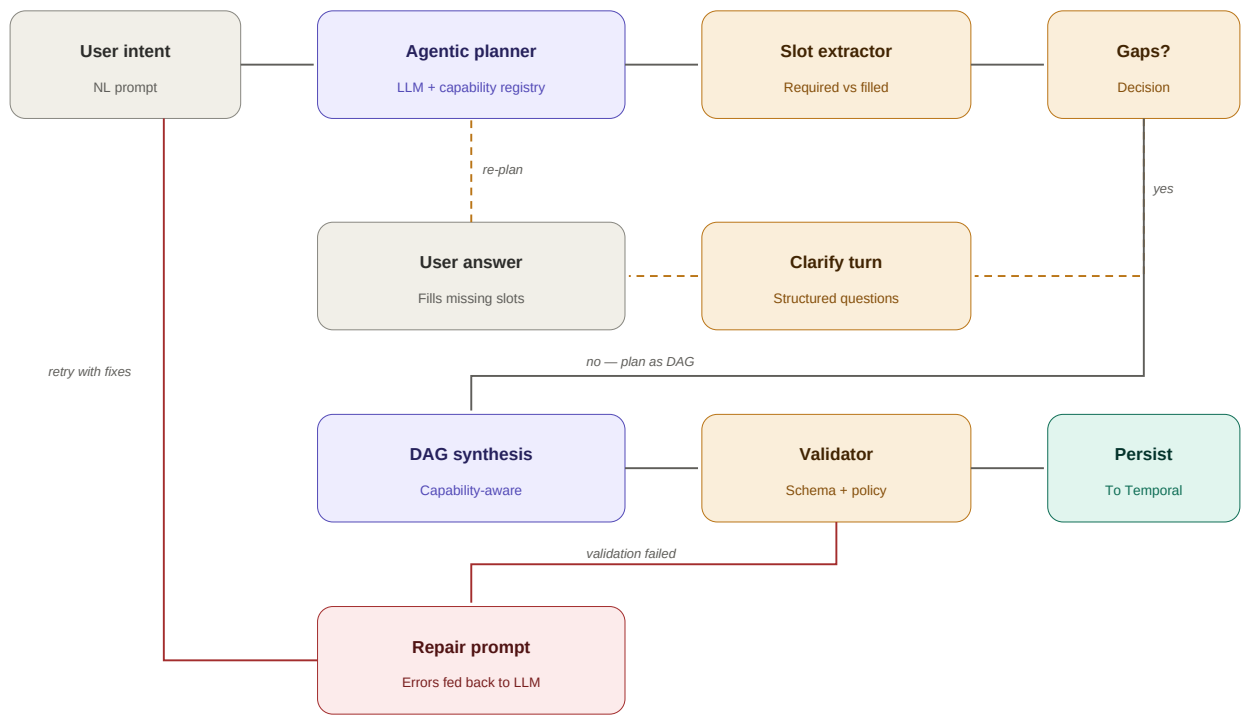
3.1 Agentic NL planning layer

The planning layer is agentic, not a single LLM call. It is a bounded loop with three exit conditions: a validated DAG, a structured failure with reason, or exhausted clarification budget. The loop must never produce code or execute anything — its sole output is a typed DAG referencing registry entries.

Behavioral requirements:

- **FR-A1.** When user intent is missing a required slot for any candidate capability, the planner **MUST** issue a clarification turn rather than fail or hallucinate.
- **FR-A2.** Clarification questions **MUST** be structured (slot id, prompt, optional enum/choices) and **MUST** be presented one cluster per turn (max 5 questions per turn).
- **FR-A3.** The planner **MUST** retain partial state across clarification turns — previously-filled slots are not re-asked.
- **FR-A4.** When a draft DAG fails validation, the validator’s structured errors **MUST** be fed back to the LLM for repair (up to MAX_REPAIR_TURNS).
- **FR-A5.** The planner **MUST** refuse to produce a DAG that references a capability not in the tenant’s allowlist.
- **FR-A6.** Every plan **MUST** be presented to the user for confirmation before execution; the user can edit slot values directly without re-prompting the LLM.
- **FR-A7.** The planner’s system prompt and the registry digest used for that plan **MUST** be recorded in the audit trail (deterministic replay).

Agentic loop diagram:



Loop bounded by MAX_CLARIFY_TURNS and MAX_REPAIR_TURNS — exceed either, return structured failure

Diagram 1 — the planner loops between extraction, clarification, synthesis, and validation. Each loop edge has a budget; exhausting any budget returns a structured failure to the user with the partial state preserved for manual completion.

3.2 Capability registry & version control

Capabilities are first-class, versioned artifacts. The registry is the contract between the LLM (which proposes them), the validator (which checks them), and the runtime (which executes them). Registry updates follow a controlled lifecycle — this is the change-management surface for the entire platform.

Registry requirements:

- **FR-R1.** Every capability has a stable name, an owner team, and a list of versions.
- **FR-R2.** Every version is uniquely identified by SemVer and bound to a specific OCI image digest plus a Sigstore signature.
- **FR-R3.** Versions have lifecycle states: *draft* → *active* → *deprecated* → *retired*. Only active versions are eligible for new plans; deprecated versions still execute but emit warnings; retired versions are blocked.
- **FR-R4.** Every parameter has a JSON Schema type, required/optional flag, default, validator (regex or CEL expression), sensitivity flag, and an *elicit_prompt* used by the agentic planner when the slot is missing.
- **FR-R5.** Each version declares required permissions (scopes) and IO schemas (input types it consumes, output types it produces). The validator uses IO schemas to type-check edges in the DAG.
- **FR-R6.** Tenants MAY pin capabilities to specific version constraints (semver range). Defaults to *latest active*.
- **FR-R7.** Registry mutations are append-only and audited; nothing is ever deleted, only state-transitioned.
- **FR-R8.** The runtime constraint of a version (compatible agent runtime semver) MUST be enforced — an out-of-date agent rejects the task.

Data model:

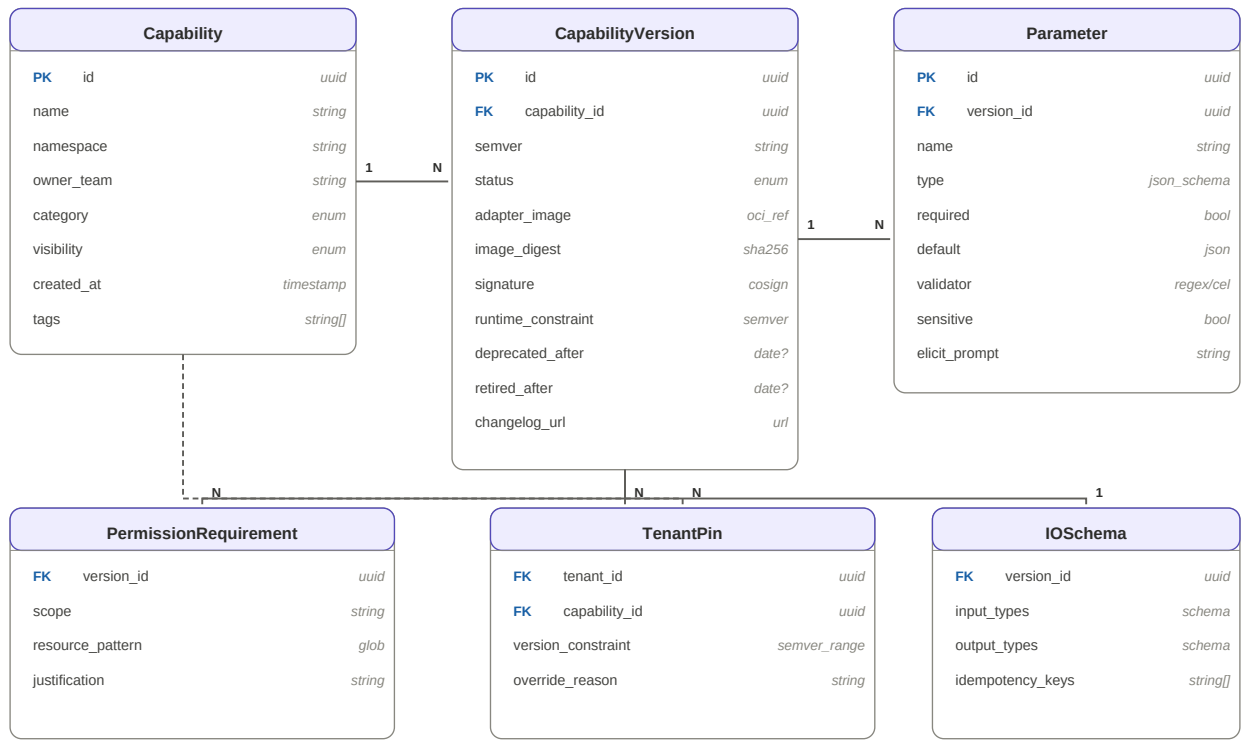


Diagram 2 — entities and relationships in the capability registry. TenantPin lets enterprise customers freeze versions for change-control. PermissionRequirement and IOSchema are what make the validator strict.

3.3 DAG construction is capability-aware

The LLM never invents a step. The planner constructs the DAG by selecting capabilities from the registry filtered by tenant allowlist, mapping NL phrases to capability names via embedding similarity over capability descriptions and tags, and binding parameters according to each capability's schema. Edges between nodes are inferred by matching upstream output types to downstream input types using the IOSchema; ambiguous edges trigger clarification.

3.4 Multi-tenancy

- **FR-T1.** Every API call carries a verified tenant context; no operation crosses tenant boundaries.
- **FR-T2.** Postgres uses row-level security with `tenant_id` session GUC; broker topics are tenant-partitioned with ACLs; object storage uses per-tenant bucket prefixes (or dedicated buckets at enterprise tier).
- **FR-T3.** Each tenant has a CMEK key for envelope encryption of artifacts and secrets.
- **FR-T4.** Per-tenant rate limits, concurrency caps, and LLM token budgets are enforced at the gateway and the LLM gateway.
- **FR-T5.** Audit, logs, metrics, and traces are tagged with `tenant_id`; tenant-scoped dashboards are available to tenant admins.

3.5 Notifications & feedback

- **FR-N1.** Workflows emit lifecycle events: queued, started, step-started, step-failed-retrying, step-failed-permanent, completed, compensated.
- **FR-N2.** Notifications support email, Slack, Teams, webhooks, and a status API (REST + SSE).
- **FR-N3.** Each tenant configures channel preferences per workflow class; per-recipient allowlists prevent abuse.
- **FR-N4.** Undelivered notifications go to a DLQ visible to tenant admins.

4. Non-functional requirements & SLAs

4.1 Service level objectives

These are the public-facing SLAs at the Enterprise tier. Standard tier targets are 1 nine looser on availability and 2× the latency budgets.

Metric	Enterprise SLA	Internal SLO	Notes
Control plane availability	99.9%	99.95%	API gateway + planner
Data plane availability	99.95%	99.99%	Bus + orchestrator (pull-based, agents tolerate brief outages)
NL plan p95 latency	< 5 s	< 4 s	Excludes user clarification turns
NL plan p99 latency	< 12 s	< 10 s	
Step dispatch p95	< 500 ms	< 300 ms	From task ready to agent receive
Step dispatch p99	< 2 s	< 1.5 s	
Workflow durability	99.999%	99.9999%	No lost workflows after acknowledgment
RPO (data loss)	≤ 60 s	≤ 30 s	Synchronous replication
RTO (recovery)	≤ 15 min	≤ 10 min	Warm-standby region
Throughput per region	10k workflows/min	20k workflows/min	Burst-tested
Tenant isolation incidents	0	0	Hard requirement

4.2 Security requirements

- **NFR-S1.** Zero-trust east-west: mTLS via service mesh, no implicit network trust.
- **NFR-S2.** Adapters run in sandboxed execution (gVisor minimum, Firecracker microVMs for tenant-uploaded adapters in v3) with seccomp, dropped capabilities, read-only root, no-new-privileges.
- **NFR-S3.** All adapter images are signed (Sigstore/cosign) with SLSA L3 provenance; agents verify before execution.
- **NFR-S4.** Per-tenant CMEK with quarterly rotation; secrets vended just-in-time with TTL ≤ 15 min.
- **NFR-S5.** Egress allowlists per capability; default-deny on agent network egress.
- **NFR-S6.** Append-only audit log to a write-only sink; tamper-evident (hash-chained).
- **NFR-S7.** Prompt-injection defenses: instruction isolation, structured output enforcement, registry-restricted output, never treating user content as system instructions.
- **NFR-S8.** Compliance: SOC 2 Type II, ISO 27001, GDPR. HIPAA-ready posture (BAA available at Enterprise tier).

4.3 Scalability targets

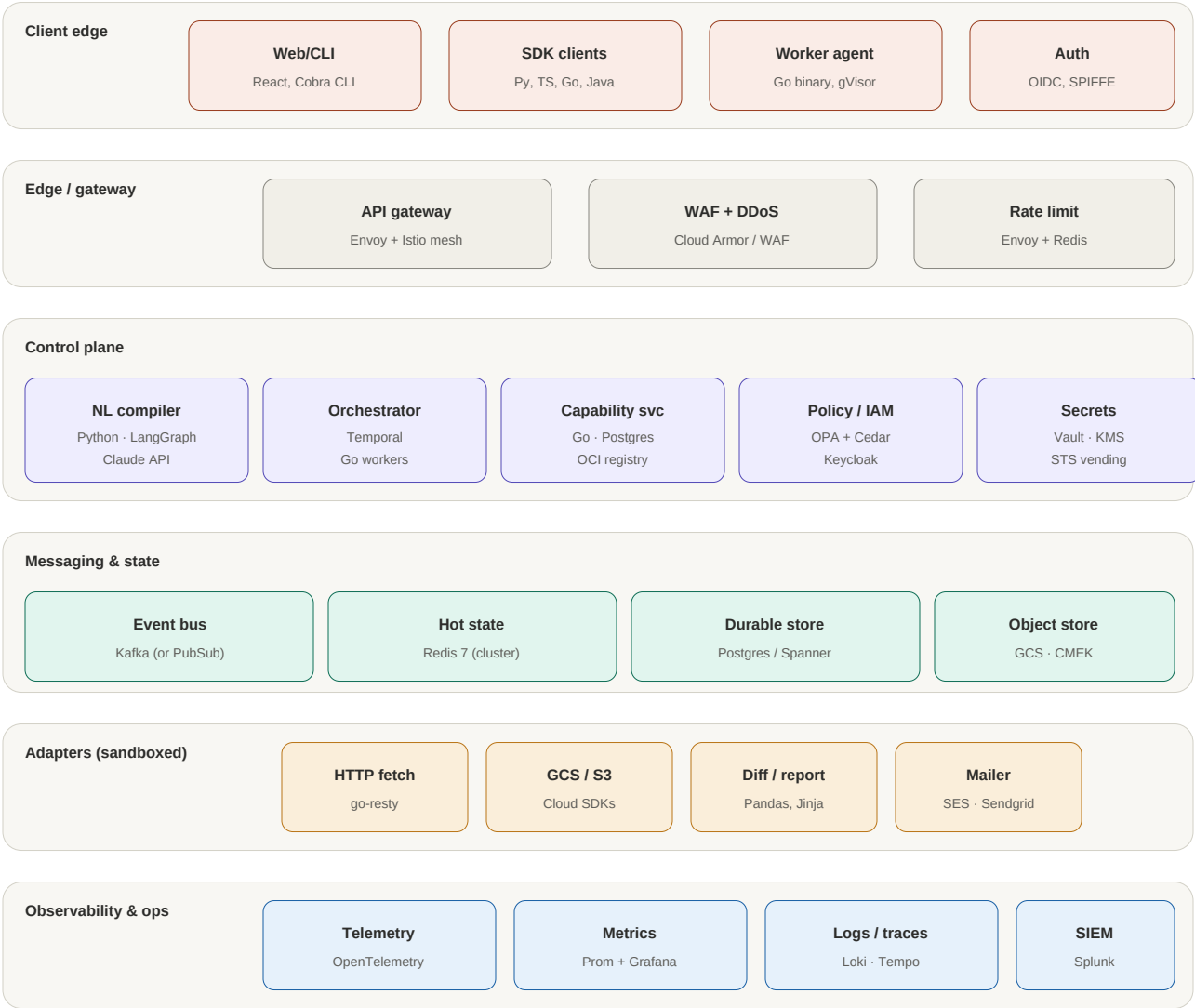
- Linear horizontal scaling of orchestrator workers, planner replicas, and adapter pool.
- 10,000 active tenants per region with no cross-tenant noisy-neighbor effects.
- 100,000 concurrent in-flight workflows per region.
- 1M workflow steps/hour aggregate throughput per region at design point.
- Agent fleet: tested with 50,000 connected agents per region (long-poll connections).

4.4 Cost guardrails

LLM token spend is the single largest variable cost. Each tenant has a daily token budget enforced at the LLM gateway; planner caches identical (tenant, intent, registry-digest) tuples for 24h. Embedding lookups in the registry are precomputed and refreshed on every registry mutation.

5. Technology stack

5.1 Stack overview



CI/CD: GitHub Actions · Argo Rollouts. IaC: Terraform · Helm. Runtime: Kubernetes (GKE/EKS), multi-zone, blue-green deploys.

Diagram 3 — layered tech stack. The control plane runs on Kubernetes with blue-green deployments. State of record is Postgres for relational data and GCS (or S3) for artifacts. Hot caches use Redis. The bus is Kafka.

5.2 Message broker decision

The bus is the spine of the data plane. Three candidates were evaluated: Apache Kafka (or its managed equivalents Confluent Cloud / AWS MSK / Google Pub/Sub), Redis (Streams + cluster), and Aerospike. They are not interchangeable — each is built for a different access pattern.

Property	Kafka / Pub/Sub	Redis Streams	Aerospike
Primary use	Durable log + queue	Hot ephemeral msgs + cache	Low-latency KV / record store
Durability model	Replicated commit log, configurable acks	AOF + cluster replicas	Hybrid SSD + replication
Ordering	Per-partition strict ordering	Per-stream ordering	N/A (KV semantics)
Throughput / node	~1M msg/s	~1M ops/s	~1M ops/s with sub-ms p99
Latency p99	~10-50 ms	~1-5 ms	< 1 ms
Multi-tenant isolation	Topics + ACLs + quotas (mature)	Per-key prefix; quotas weaker	Namespaces + sets; mature
Retention / replay	Hours to forever; rewindable	Hours; trim-based	TTL-based; not log-shaped
Consumer groups	Native, durable offsets	Yes (XREADGROUP)	Not natively a queue
Pull-based long-poll	Native	Native	Polling client; not idiomatic
Transactions	Exactly-once with txn API	MULTI/EXEC, weaker	Strong single-record
Ops cost	High (ZK/KRaft, brokers, ops)	Low to medium	Medium (clustering, XDR)
Ecosystem fit	Excellent (Temporal, OTel, Connect)	Good for caches, weaker for workflows	Niche; great when you need it

Decision

Kafka (or Google Pub/Sub at GCP-first deployments) is the event bus. It is the only candidate built for durable, partitioned, replayable, tenant-isolated message logs. Per-tenant topics with per-workflow partition keys give us strict per-workflow ordering with horizontal scale. Replay enables audit-grade reconstruction.

Redis is the hot-state cache, not the bus. Idempotency keys, rate-limit counters, planner prompt cache, ephemeral session state, dispatch hints. Sub-millisecond reads matter here; durability does not.

Aerospike is not selected for v2. It excels at sub-millisecond key-value workloads with hybrid memory/SSD economics, but it is not a message broker and using it as one means re-implementing offsets, consumer groups, and replay on top of records. It is a candidate for the idempotency cache at very high scale in v3 if Redis hits headroom limits, but bringing in another data system in v2 adds operational surface without addressing a v2 bottleneck.

5.3 Orchestration: Temporal

Temporal is the workflow orchestrator. The selection rationale:

- Workflow-as-code (Go/Python/Java) gives us conditionals, loops, signals, timers, and saga compensation natively — far more expressive than YAML DAG engines.
- Built-in durable execution: state machine survives orchestrator restarts; replay reconstructs history deterministically.
- Heartbeats and per-activity retry policies; integrates cleanly with our retry/backoff requirements.

- Multi-namespace = clean tenant isolation at the orchestrator layer.
- Mature operator tooling, observable, well-supported. Self-hosted or Temporal Cloud.

Temporal sits between the planner and the bus. The planner produces a DAG; a thin shim translates that DAG into a Temporal workflow execution that calls *activities* — activities publish task messages onto Kafka and await completion signals from the bus. This keeps Temporal as the source of truth for workflow state without putting it on the worker-agent execution path.

5.4 Other choices & rationale

Layer	Choice	Why
LLM provider	Anthropic Claude (multi-tenant via API gateway)	Strong tool-use, structured output, instruction adherence; Bedrock fallback for AWS-resident tenants.
NL planner runtime	Python + LangGraph	State-machine framework purpose-built for agentic loops with budget control.
Worker agent	Single Go binary	Static binary, small footprint, easy deployment to mixed environments.
Sandbox	gVisor (default), Firecracker (high-isolation tier)	Defense-in-depth without paying microVM cost on every invocation.
Policy engine	OPA (Rego) + Cedar for ABAC	OPA mature, Cedar concise for permission rules; both proven at scale.
Secrets / KMS	HashiCorp Vault + cloud KMS	Vault for app-level dynamic secrets, KMS for envelope encryption.
Identity	Keycloak (OIDC) + SCIM, SPIFFE for workload identity	Standards-based; Enterprise customers can federate their IdP.
Observability	OpenTelemetry → Prom + Loki + Tempo + Grafana	OTel-first, pluggable backends, every signal carries tenant_id.
IaC	Terraform + Helm; Argo Rollouts	Declarative infra, GitOps deployments, progressive rollouts.

6. Client-side requirements

The customer is responsible only for running the worker agent in their network (and optionally the SDK in their applications). Everything else is SaaS. The agent is the security boundary — their data does not leave their environment.

6.1 Worker agent host

Requirement	Specification	Notes
OS	Linux (kernel ≥ 5.4) or Windows Server 2019+ (v2.5)	Linux only at v2 GA; Windows in v2.5
CPU	≥ 2 vCPU (4 vCPU recommended)	Per agent; scale out for parallel workloads
Memory	≥ 4 GB (8 GB recommended)	Sandbox overhead included
Disk	≥ 20 GB free; SSD strongly preferred	Adapter image cache + temp working dirs
Container runtime	containerd, Docker, or Podman	Required for adapter sandboxing
Time sync	NTP within ± 5 s of UTC	Required for credential validity
DNS	Egress DNS to public (or split-horizon to private resolver)	Adapter image registry + bus DNS
Hypervisor support	KVM extensions enabled (high-isolation tier only)	For Firecracker microVM sandbox

6.2 Network

- **Outbound only.** Agent makes outbound TLS (port 443) to *bus.{vendor}.com* and *registry.{vendor}.com*. No inbound firewall holes required.
- Optional: outbound to customer-internal endpoints used by adapters (HTTP source URLs, SMTP relays, S3/GCS, etc.). These are governed by the egress allowlist enforced at the agent.
- Proxy support: HTTP/HTTPS proxy with auth (basic, NTLM via spn), and SOCKS5.
- Bandwidth: 1 Mbps sustained per active workflow; bursts to 100 Mbps for large file transfers.

6.3 Identity & auth

- **Agent enrollment.** One-time bootstrap token (printed during agent install) exchanged for a SPIFFE workload identity. Subsequent auth is mTLS with rotating workload SVID.
- **User auth.** OIDC against the customer's IdP (Okta, Azure AD, Google Workspace, generic OIDC) at Enterprise tier; built-in IdP at Standard tier. SCIM 2.0 for provisioning.
- **Session.** Short-lived access tokens (15 min) + refresh; tokens scoped per tenant + role.

6.4 SDKs (optional, for programmatic submission)

First-class SDKs in Python, TypeScript, Go, and Java. Each SDK exposes: *submit_intent(text)*, *plan(text)* (returns the DAG without executing), *execute(plan)*, *watch(workflow_id)* (SSE stream), *cancel*, *list_capabilities()*. SDKs handle auth refresh, retries, and idempotency keys.

6.5 Customer onboarding checklist

- Provision tenant in the SaaS console (self-serve at Standard tier; account-managed at Enterprise).
- Configure IdP federation (Enterprise) or invite first admin (Standard).
- Define recipient allowlists (email domains, Slack channels) and egress allowlists (URL patterns).

- Install one or more agents in the network(s) where workflows must execute.
- Bind cloud credentials (GCS/S3/SES) via Vault dynamic secrets or BYOK service-account keys.
- Submit first workflow; confirm plan; observe in dashboard.

7. Algorithms & data structures (pseudocode)

The shapes below define the contracts between the planner, validator, registry, and orchestrator. They are intentionally language-agnostic and free of implementation detail.

7.1 Capability data structure

```
Capability {
  id : uuid
  name : string // e.g. "http.fetch"
  owner : team
  versions : list<Version>
}

Version {
  semver : string // e.g. "1.4.0"
  status : enum {draft, active, deprecated, retired}
  parameters : list<Param>
  io_schema : { in: JsonSchema, out: JsonSchema }
  permissions : list<PermissionScope>
  adapter_image : oci_ref // signed digest
  runtime_constraint : semver_range
  idempotency_keys : list<param_path> // which params hash into key
}

Param {
  name : string
  type : JsonSchema // typed, validated
  required : bool
  default : any?
  validator : regex | cel_expr
  sensitive : bool // redact in logs
  elicit_prompt : string // shown to user when missing
}
```

7.2 Agentic planning loop

```
function plan(intent: string, tenant: TenantCtx) -> PlanResult:
  state := { intent, slots: {}, history: [], turn: 0 }
  registry := load_registry(tenant.allowlist, tenant.pins)

  loop:
    state.turn += 1
    if state.turn > MAX_CLARIFY_TURNS + MAX_REPAIR_TURNS:
      return failure("budget_exhausted", state)

    output := llm.compile(
      intent = state.intent,
      slots = state.slots,
      tools = registry,
      mode = STRUCTURED_DAG_OR_QUESTIONS
    )

    match output.kind:
      case QUESTIONS: // missing slots
        answers := ask_user(output.questions)
        state.slots.merge(answers)
        continue
      case DAG:
        errors := validate(output.dag, registry, tenant)
```

```
if errors.empty:  
    return present_for_confirmation(output.dag)  
state.intent += repair_prompt(errors) // self-correct  
continue
```


7.3 DAG validation

```
function validate(dag: DAG, registry, tenant) -> list<Error>:
  errors := []

  // 1. Structural
  if has_cycle(dag): errors += "cycle"
  if dag.nodes.empty: errors += "empty_dag"

  for node in dag.nodes:
    // 2. Capability resolves and is active
    cap := registry.resolve(node.name, node.version_constraint, tenant)
    if cap is null: errors += (node, "unknown_capability")
    if cap.status == retired: errors += (node, "retired")

    // 3. Parameters bound and typed
    for p in cap.parameters:
      v := node.params.get(p.name)
      if p.required and v is null: errors += (node, p, "missing")
      if v is not null and not type_match(v, p.type):
        errors += (node, p, "type_mismatch")
      if v is not null and not validator_passes(v, p.validator):
        errors += (node, p, "validator_failed")

    // 4. Tenant has every required permission
    for perm in cap.permissions:
      if not tenant.policies.allow(perm, node.params):
        errors += (node, perm, "denied")

    // 5. Edge type compatibility
    for edge in dag.in_edges(node):
      upstream := dag.node(edge.from)
      if not schema_subtype(upstream.cap.io.out, cap.io.in):
        errors += (edge, "io_incompatible")

  return errors
```

7.4 Version resolution

```
function resolve(name, constraint, tenant) -> Version | null:
  all := registry.versions(name)
  allowed := all where v.status in {active, deprecated}
  allowed := allowed where tenant.allowlist.permits(name)
  allowed := allowed where semver_match(v.semver, constraint)

  // Tenant pin wins if compatible
  pin := tenant.pins.get(name)
  if pin and pin in allowed: return pin

  // Otherwise: highest active > highest deprecated
  active := allowed where v.status == active
  if active.nonempty: return max_by_semver(active)
  return max_by_semver(allowed) // deprecated only
```

7.5 Idempotency at the adapter boundary

```
function execute_step(task) -> Result:
  cap := registry.resolve(task.cap_name, task.version, task.tenant)
  key_inputs := pick(task.params, cap.idempotency_keys)
  key := sha256(task.workflow_id + task.step_id + canonical(key_inputs))
```

```
cached := redis.get("idem:" + key)
match cached:
  case completed: return cached.result // safe replay
  case in_flight: wait_with_timeout(cached.lease)
  case null: pass // proceed

redis.set("idem:" + key, in_flight, ttl=2*task.deadline)
try:
  creds := vend_credentials(cap.permissions, ttl=15min)
  result := sandbox.run(cap.adapter_image, task.params, creds)
  redis.set("idem:" + key, completed(result), ttl=24h)
  return result
catch e:
  redis.delete("idem:" + key) // allow retry
  raise classify_error(e) // retryable / permanent
```

7.6 Retrv classification

```
function classify_error(e) -> ErrorClass:
  match e:
    case NetworkTimeout : retryable(backoff=exp, max_attempts=5)
    case Http5xx : retryable(backoff=exp_jitter, max=5)
    case Http429 : retryable(honor_retry_after_header)
    case Http4xx : permanent(reason="client_error")
    case AuthExpired : retryable_after_creds_refresh
    case PermissionDenied : permanent(reason="policy")
    case ResourceExhausted : retryable(backoff=long, max=3)
    case ValidationFailed : permanent(reason="input")
    default : permanent(reason="unknown")
```

7.7 Tenant-partitioned dispatch

```
// Topic naming
tasks.<tenant_id>.v1
status.<tenant_id>.v1
dlq.<tenant_id>.v1

// Partitioning: key = workflow_id ensures per-workflow ordering
// without serializing across the tenant.
publish(
  topic = "tasks." + task.tenant_id + ".v1",
  key = task.workflow_id,
  value = sign(task, tenant_kek),
  headers = {
    tenant_id, workflow_id, step_id, attempt,
    deadline_ms, idempotency_key, trace_id
  }
)
```

8. Risks & open questions

Risk	Likelihood	Mitigation
LLM produces invalid DAGs at high rate, hurting acceptance	Medium	Strict structured output, repair loop, capability embeddings tuned per registry, A/B harness on planner prompt versions.
Prompt-injection through user content (file names, URLs)	Medium	Instruction isolation; user content never treated as instructions; adapter-level egress allowlists; sandbox; output schema validation.
Capability registry growth pollutes context window	Medium-High	Embedding-based retrieval over registry; only top-K relevant capabilities placed in planner context.
Kafka operational burden	Medium	Use Confluent Cloud / MSK at v2 to defer self-hosting; revisit at scale.
Worker agent compromise pivots to customer network	Low-Medium	Sandbox; outbound-only; egress allowlists; vended creds with TTL; agent runs unprivileged; SLSA-verified images.
LLM cost growth dominates margin	High	Per-tenant token budgets; planner cache; smaller model on warm cache hits; embedding-precomputation.
Customer agent fleets out of date, breaking new capabilities	Medium	Runtime constraint enforcement; auto-update channel (opt-in); deprecation windows announced 90d in advance.

Open questions for v2 review

- Do we ship Pub/Sub-only at GCP launch, or unify on Kafka via Confluent Cloud across clouds from day one?
- Default sandbox: gVisor for everyone, or Firecracker for Enterprise tier and gVisor for Standard?
- Where does NL clarification UI live — chat-style modal in the web app, or async via email/Slack for batch users?
- How do we surface registry version-drift warnings to tenant admins (banner? email digest?)
- Should tenant-pinned versions block automatic security patches, or only feature changes?